Crewmate

Technical handbook for the review: what the app is, how it is built, where everything lives, how data flows and how it is deployed.

Fynd × NMIMS Hyderabad hackathon, case C35 (Scope Change Desk). Repository github.com/Krishnass05/fynd, branch `krishna`. Live: https://crewmate-sable.vercel.app. Generated from the code on 2026-10-18. All data is synthetic. A workflow aid, not legal advice.

Numbers at a glance: 25 pages, 43 API routes, 46 browser API functions, 35 components, 13 rule modules with 92 unit tests, an end-to-end smoke test, 81 recorded decisions.

Contents

1. The product in one page
2. Tech stack and every technical term
3. Architecture: how a click becomes saved data
4. Frontend: where it is and how it is organised
5. Every page (URL to file)
6. Every button (label, file, line, what it does)
7. Backend: API routes, services and rules
8. Every API route (URL, method, file, server function)
9. Browser API functions (lib/api.ts)
10. Database: Supabase, tables and the data schema
11. How the frontend is linked to Supabase
12. Login, sessions and Google sign-in
13. AI features and their fallbacks
14. Business rules and the three judged checks
15. Deployment (Vercel + Supabase) and environment variables
16. Testing
17. What has been built (feature list)
18. Known limits and honest answers
19. Team ownership
20. Viva quick answers

1. The product in one page

Crewmate is a freelance marketplace (like Upwork) built around one problem: *scope creep*, small unpaid extra asks that pile up. A client hires a freelancer for a job; every extra request is compared with the agreed scope, priced, and only becomes work once both sides agree.

> **The journey:** client posts a job → freelancers apply → client hires one (price split into milestones) → **client uploads the SOW** → freelancer reviews it in a table, fills in hours, approves (client confirms changed hours) → signed scope → rows become **tasks** → freelancer works, posts a **daily check-in**, marks tasks done → client approves them → **progress bar** (approved hours ÷ agreed hours) → milestones become ready → client approves milestones. At any time the client can **ask for something new in plain English**: the rules label it (included, excluded, new, defect, unclear), the freelancer accepts as included, fixes free as own defect, quotes, asks for details or declines, and a quote is **negotiated** (counter offers) until one side accepts. Chat and voice/video calls hide phone numbers and emails (also when spelled out in words).

SOW = Statement of Work, the written agreement: what is included, what is excluded, price, deadline. **Scope** = the fence around the job. **Change order** = a priced, approved extra. **Utilization** = committed hours ÷ available hours.

2. Tech stack and every technical term

Term	What it is	Where / how we use it
Next.js 16 (App Router)	A React framework that serves pages and API endpoints from one project.	Pages in `app/**/page.tsx`; API in `app/api/**/route.ts`. Run with `npm run dev`.
React 19	The UI library: components re-draw when their data changes.	Every page and component (`.tsx`).
TypeScript	JavaScript with types, checked before running.	Pages, components, server (`lib/server`), shared types (`lib/types.ts`). Checked with `npx tsc --noEmit`.
Plain JavaScript rules engine	The money and workflow rules, with no framework and no dependencies, so they are easy to test.	`lib/engine/*.js`, each with a `.d.ts` type file and a `.test.js`.
node:test	The test runner built into Node.js.	`npm test` runs all engine tests.
API route / endpoint	A URL the browser calls to read or change data, returning JSON.	See section 8.
JSON	The text format data travels in.	Every API request and reply; the whole database row is JSON.
REST	Using URLs + HTTP methods (GET read, POST change) for data.	Our API, and how the server talks to Supabase (PostgREST).
Supabase	Hosted Postgres database with a REST interface.	Stores all app data (section 10).
Postgres / jsonb	The SQL database; jsonb is a column type that holds JSON.	`app_state.data` is jsonb.
Row Level Security (RLS)	Postgres rules on who may read a row.	ON with no policies: the public key can read nothing; only the server key can.
Service role / secret key	The powerful Supabase key that skips RLS.	Only on the server (env var), never in the browser or git.

Term	What it is	Where / how we use it
Optimistic concurrency	Save only if nobody else saved first ("update where version = n"), otherwise retry.	`lib/server/persist.ts`, up to 14 retries.
Vercel	Hosting for Next.js: builds from GitHub, runs API routes as serverless functions.	Production branch `krishna`; live at crewmate-sable.vercel.app.
Serverless function	Server code that starts on demand for each request (no always-on server).	Each API route on Vercel.
Environment variable	A setting/secret given to the server outside the code.	Section 15.
Cookie / session	Small data the browser sends back each time; the session cookie says who you are.	`cd_session`, HTTP-only, signed with HMAC.
HMAC	A signature using a secret, so a cookie cannot be forged.	`lib/server/session.ts` with SESSION_SECRET.
scrypt	A slow password-hashing method that resists guessing.	Passwords are stored only as salted scrypt hashes.
OAuth 2.0	The standard behind "Continue with Google".	`/api/auth/google/start` and `/callback`.
AsyncLocalStorage	Node feature that keeps per-request context (who is calling, their copy of the data).	`lib/server/context.ts`, filled by `run()`.
WebRTC	Browser-to-browser audio/video.	Voice and video calls (`components/call/Call.tsx`).
STUN	A public server that helps two browsers find a route to each other.	Google STUN; no TURN server.
Signalling	Messages that set up a call (ring, offer, answer, ICE).	Stored in the `call_signals` table, polled by the browser.
Web Speech API	Browser speech-to-text.	Live captions in calls, checked by the contact guard.
Gemini	Google's language model.	Second opinion for the chat/call guard and the help assistant; rules run first.
Polling	The browser asks the server again every few seconds.	Chat every ~4 s, call signals every ~1.4 s.
CSS Modules	CSS files whose class names are private to one component.	`*.module.css` next to components.
Idempotent	Doing the same thing twice has the effect of doing it once.	Re-uploading a SOW or re-asking the same request does not double-charge (check 3).
Smoke test	An end-to-end test that calls the real API in order.	`scripts/smoke.mjs`, `npm run smoke`.

3. Architecture: how a click becomes saved data

```
Browser (React page / component)
   │  calls a function in lib/api.ts   e.g. actOnTask(projectId, taskId, {action:'accept'})
   ▼  fetch → HTTPS → JSON   (+ cd_session cookie; demo accounts also send x-demo-user)
Next.js API route   app/api/projects/[id]/tasks/[taskId]/route.ts   (a Vercel serverless function)
   │  run() in lib/server/http.ts: parse body, find the user, wrap in withState(), turn errors into JSON
   ▼
Service   lib/server/service.ts   (role checks: is this user in the project? client or freelancer?)
   │  uses the rules in lib/engine/*.js  (negotiation, scopeDesk, progress, guard ...)
   ▼
Data   db() = this request's copy of the whole app state
   │  persist.ts: load row 'main' from Supabase at the start; at the end, if anything changed,
   │  PATCH app_state?id=eq.main&version=eq.N  → if someone saved first, re-run the request (≤14 tries)
   ▼
Supabase (Postgres)  table app_state (one jsonb row) + table call_signals
```

Without Supabase keys (development) the same code keeps the data in server memory instead; it resets on restart. Reads (GET) never write. The browser never talks to Supabase directly.

4. Frontend: where it is and how it is organised

- **`app/`**: pages (one folder per URL). `(client)` and `(freelancer)` are route groups: they share a layout (top bar, menus) but do not appear in the URL.
- **`components/`**: reusable parts (shell/navigation, chat, call, SOW, tasks, work, milestones, landing page, UI kit).
- **`lib/api.ts`**: the ONLY place pages talk to the server (no `fetch` in pages).
- **`lib/session.ts`, `lib/useUser.ts`**: who is signed in on this browser; a page guard sends people to /login.
- **`app/globals.css`**: colour tokens and shared styles (white background, charcoal text, orange #F37021 accents).
- **`public/`**: images and the sample SOW. `app/icon.svg` is the favicon.

Components

File	What it does	Owner
`components/auth/Parts.tsx`	Small pieces for the sign-up and log-in pages: a text field with its error message, a password field with show/hide, the strength meter, and the Google button.	Krishna
`components/brand/Logo.tsx`	The Crewmate logo: an orange "C" with a dot. The same drawing is app/icon.svg (the favicon). Decorative next to the word "Crewmate"; give it a title only when it stands alone.	Krishna
`components/call/Call.tsx`	Voice and video calls between the client and the freelancer, inside the project. Media goes browser to browser (WebRTC); our server only passes the signalling and checks the captions. While a call is on, each person's browser turns their speech into text (the browser's speech recognition) and sends every line to the server, where the contact guard checks it: if a phone number, email, link or handle is said, even in words, the call is ended for both people. Calls need Chrome or Edge (speech recognition), otherwise they do not start. A public STUN server is used to find a route; strict networks may need a TURN server.	Krishna

File	What it does	Owner
`components/changes/ChangeList.tsx`	Priced change requests with flags and approve / reject / reopen. Props: projectId, changes, canDecide (client only), onChanged(project). Used on both sides.	Harini
`components/chat/Chat.tsx`	Project chat between the client and the freelancer. (brought onto krishna by Krishna). Used by both project pages. Props: projectId (required) and role (optional: 'client' | 'freelancer'). Messages are saved on the server; the server hides email addresses and phone numbers before saving.	Harini
`components/dashboard/ClientDashboardView.tsx`	Client dashboard: number tiles, recent activity, what needs the client, projects and jobs. Shown on /client/dashboard, opened from the avatar menu. (look).	Sanjana
`components/fx/BlurText.tsx`		
`components/fx/CountUp.tsx`		
`components/fx/PillNav.jsx`		
`components/fx/SpotlightCard.tsx`		
`components/help/HelpBot.tsx`	Help assistant: a small chat on the left of the screen that answers questions from clients and freelancers about how this site works. With an AI key set, a model writes each reply but only from the written guide (lib/engine/helpbot.js); without one, or if the model is down, the keyword answer from the same guide is used. The panel says which one is answering.	Krishna
`components/home/MarketHome.tsx`	The first page of each side: a headline, a short description and a big search box in the middle, popular searches, then reviews as a coverflow carousel. The same layout serves clients (search freelancers) and freelancers (search jobs). (look follows the joveo.com home page).	Krishna
`components/landing/CategoryCarousel.tsx`	"Find freelancers for every type of work": a 3D coverflow of the ten categories. The active slide is large and flat, its neighbours turn away, and a colour wash sweeps to the active category's tone. Our own build (the paid React Bits Pro Prism Carousel needs a licence we do not have). Arrow keys, dots and clicking a side slide move it; clicking the active slide (or Enter) opens the search for that category. No autoplay.	Krishna
`components/landing/Hero.tsx`	The cinematic hero: full-bleed background (video when supplied, otherwise crossfading photos) under a dark overlay, the headline, the Hire / Work toggle and the search. The pause button stops the slideshow (and video). With reduced motion it shows one still frame.	Krishna

File	What it does	Owner
`components/landing/HeroSearch.tsx`	The big hero search with autocomplete. Suggestions come from local sample data (domains, topics and sample services) through `suggest`, kept separate so it can later call an API. Keyboard: arrows move through suggestions, Enter picks one (or searches the text), Escape closes.	Krishna
`components/landing/LandingNav.tsx`	Top bar of the public home page: Crewmate wordmark, Find Talent, Find Work, Explore Categories (drop-down), How It Works, and on the right Log in and Sign up, which open /login and /register. Sticky; on phones the links move into a drawer.	Krishna
`components/landing/Sections.tsx`	The rest of the home page: How it works (two journeys), sample talent, the final call to action and the footer. All people and text are sample content for the demo.	Krishna
`components/landing/Segmented.tsx`	A two-option pill toggle with a sliding indicator. Radio-group pattern: arrow keys move the choice, only the chosen option is in the tab order. `tone="glass"` is for the dark hero, `tone="light"` for white sections. Used by the hero (Hire / Work) and How it works.	Krishna
`components/milestones/MilestoneList.tsx`	The milestones of a project, with who can act on each. Props: projectId, milestones, summary, role, onChanged(project). The freelancer marks one ready; the client approves it. Record only: no money moves.	Krishna
`components/milestones/MilestoneSplit.tsx`	Split the agreed price into milestones before accepting an applicant. Props: price (whole rupees), onConfirm(parts), onCancel(), busy. The parts must add up to the price exactly.	Krishna
`components/profile/ClientProfileView.tsx`	A client's profile: who they are, company details, rating and reviews, and what they have posted. Shown to the client (their own page) and to freelancers (from a job page).	Krishna
`components/profile/Parts.tsx`	Small pieces shared by the freelancer and client profile pages: star rating, review list, previous work, and the summary box. All review and work content is sample data and is labelled that way.	Krishna
`components/reviews/ReviewCarousel.tsx`	A coverflow carousel for the client reviews: the middle card is large and flat, the others turn away to the sides, and a soft colour wash sweeps to the tone of each new card. Our own plain-CSS version (the paid React Bits Prism Carousel is not used). Arrow keys, buttons and dots work; it pauses on hover and focus and does not move on its own for people who prefer reduced motion.	Krishna
`components/shell/AppNav.tsx`	The navigation bar used on every page: React Bits PillNav in our colours. (look). It highlights the page you are on, uses Next links (no full reload), and folds into a menu button on narrow screens.	Sanjana

File	What it does	Owner
components/shell/AppShell.tsx	The frame around every signed-in page: a top navigation bar with drop-down menus (Jobs, Services), a search box and the user, plus the PillNav menu on phones. (look). Links come from lib/nav.ts; the Services menu comes from lib/services.ts.	Sanjana
components/sow/SowReview.tsx	The first SOW as a table, and its sign-off. Used on both project pages. Client: uploads it (or a new version), and confirms or sends back the freelancer's hours. Freelancer: fills in or changes the hours per row and approves, or sends it back with a reason. The rules are on the server (reviewSow in lib/server/service.ts); this only shows and sends.	Krishna
components/sow/SowUpload.tsx	Upload a SOW (.txt or .md): the first SOW (the client uploads it, the freelancer signs it off) or an amendment. Props: projectId, kind, onDone(result). Reads the file in the browser and sends its text.	Krishna
components/tasks/TaskRequests.tsx	New tasks in plain English, and the negotiation on their price. Used on both project pages. Client: asks for something new, answers questions, counters / accepts / declines quotes, withdraws a request. Freelancer: answers each request (already included, own defect, quote, ask, decline), then counters / accepts. All rules are on the server (lib/server/service.ts, lib/engine/negotiation.js); this only shows and sends.	Krishna
components/ui/FreelancerCard.tsx	A freelancer as a card: photo block with initials and a heart to save, name, headline, rating, place, skills, rate. Used when a client browses freelancers. Profiles are sample data. (look).	Sanjana
components/ui/JobCard.tsx	A job as a pastel card: pill with the status, title, short description, budget, deadline and applicants, then a footer button. Used on the client and freelancer job lists and dashboards. (look).	Sanjana
components/ui/ProjectCard.tsx	A project as a pastel card: pill, title, a line of detail, a row of facts, a capacity bar, then a footer button. Used by the client and freelancer dashboards. (look).	Sanjana
components/ui/icons.tsx	Line icons (Lucide style, one family, 1.75 stroke). Decorative by default: aria-hidden.	
components/ui/index.tsx	Shared UI parts. She restyles these to the final design. KEEP THE EXPORT NAMES AND PROP NAMES the same so other people's pages do not break.	Sanjana
components/work/CheckIns.tsx	Daily check-ins. Used on both project pages. Freelancer: posts today's check-in (what was done, hours, blockers) and can start or finish tasks in the same step. Client: reads them, and sees a warning after 2 days without one. One check-in a day, India time (server rule).	Krishna
components/work/WorkBoard.tsx	The task list with the progress bar. Used on both project pages. Freelancer: starts tasks, marks them done, puts them in milestones. Client: approves done tasks or sends them back. Tasks come from the signed SOW and from agreed requests; the rules are on the server (lib/engine/progress.js).	Krishna

5. Every page (URL to file)

URL	File	What it shows	Uses	Owner
`/client/dashboard`	`app/(client)/client/dashboard/page.tsx`	The client dashboard (opened from the avatar menu). (look).	dashboard/ClientDashboardView	Sanjana
`/client/freelancers/:id`	`app/(client)/client/freelancers/[id]/page.tsx`	A freelancer's profile as a client sees it: header, tabs, and a hire card. Sample data. (look: Sanjana).		Krishna
`/client/freelancers`	`app/(client)/client/freelancers/page.tsx`	Browse freelancers: category and skill chips, search from the top bar, more filters, and a card grid. (look: Sanjana).	ui/FreelancerCard	Krishna
`/client/jobs/:id`	`app/(client)/client/jobs/[id]/page.tsx`	Job detail: applicants as cards, a side-by-side comparison, and Accept. Starter version.	milestones/MilestoneSplit	Sanjana
`/client/jobs/new`	`app/(client)/client/jobs/new/page.tsx`	Post a job in three steps: Details, Budget, Review. Start from a library profile to fill in the scope and see the typical price (illustrative). (look).		Sanjana
`/client/jobs`	`app/(client)/client/jobs/page.tsx`	My jobs list. Starter version: replace with the final design.	ui/JobCard	Sanjana
`/client`	`app/(client)/client/page.tsx`	The client's first page. The reviews are SAMPLE text written for the demo, not real customers.	home/MarketHome	Krishna
`/client/profile`	`app/(client)/client/profile/page.tsx`	The client's own profile: details, company, rating and reviews. The dashboard has its own page.	profile/ClientProfileView	Krishna
`/client/projects/:id/milestones`	`app/(client)/client/projects/[id]/milestones/page.tsx`	Client view of a project's milestones: see them and approve each one. (Harini can also embed <MilestoneList /> in the project page; this page works on its own.)	milestones/MilestoneList	Krishna
`/client/projects/:id`	`app/(client)/client/projects/[id]/page.tsx`	Client project page: overview, amendment upload, change orders, chat. Starter version works end to end; improve it to the final design.	chat/Chat, call/Call, changes/ChangeList, sow/SowUpload, sow/SowReview, tasks/TaskRequests, work/WorkBoard, work/CheckIns	Harini
`/client/saved`	`app/(client)/client/saved/page.tsx`	The client's saved freelancers (a shortlist). (look: Sanjana).	ui/FreelancerCard	Krishna
`/freelancer/clients/:id`	`app/(freelancer)/freelancer/clients/[id]/page.tsx`	A client's profile as a freelancer sees it (from a job page): company, rating, reviews, open jobs.	profile/ClientProfileView	Krishna

URL	File	What it shows	Uses	Owner
`/freelancer/dashboard`	`app/(freelancer)/freelancer/dashboard/page.tsx`	Freelancer dashboard: what needs attention, projects with capacity, applications, jobs to apply to.	fx/CountUp, ui/JobCard, ui/ProjectCard	Krishna
`/freelancer/jobs/:id`	`app/(freelancer)/freelancer/jobs/[id]/page.tsx`	Job detail and apply.		Krishna
`/freelancer/jobs`	`app/(freelancer)/freelancer/jobs/page.tsx`	Browse open jobs. The search box filters by words in the title, description and category; ?category= filters by domain.	ui/JobCard	Krishna
`/freelancer`	`app/(freelancer)/freelancer/page.tsx`	The freelancer's first page, laid out like the client home. The reviews are SAMPLE text written for the demo, not real freelancers.	home/MarketHome	Krishna
`/freelancer/profile`	`app/(freelancer)/freelancer/profile/page.tsx`	The freelancer's own profile, laid out like the client profile: a header card, about, skills, previous work, past clients and reviews. The edit form sits in its own section.		Krishna
`/freelancer/projects/:id`	`app/(freelancer)/freelancer/projects/[id]/page.tsx`	Freelancer project workspace: signed SOW upload, change orders, capacity, chat.	chat/Chat, call/Call, changes/ChangeList, sow/SowReview, milestones/MilestoneList, tasks/TaskRequests, work/WorkBoard, work/CheckIns	Krishna
`/auth/done`	`app/auth/done/page.tsx`	The landing after Google (or any server-side sign-in): learn who is signed in from the cookie, remember it in this browser, and go to the right side.		Krishna
`/demo`	`app/demo/page.tsx`	The three case outcome checks, in printed order.	shell/AppNav	Krishna
`/login`	`app/login/page.tsx`	Log in for clients and freelancers: email and password with validation, Google sign-in, and the demo accounts box. Wrong passwords are counted on the server and lock the email for a few minutes.	shell/AppNav	Krishna
`/`	`app/page.tsx`	The public home page (/): what a first-time visitor sees. Log in and Sign up open the start page (/start), where they pick Client or Freelancer.	landing/CategoryCarousel, landing/Hero, help/HelpBot, landing/LandingNav	Krishna
`/register`	`app/register/page.tsx`	Create an account. The same rules run here as you type and on the server, which has the final say. Email verification (Brevo) is planned: for now an account is usable straight away.	shell/AppNav	Krishna
`/start`	`app/start/page.tsx`	Start page (/start): pick Client or Freelancer, then log in. Reached from Log in / Sign up on the home page. The numbers come from our own data: no made-up user counts.	fx/BlurText, fx/SpotlightCard, shell/AppNav	Krishna

URL	File	What it shows	Uses	Owner
`/terms`	`app/terms/page.tsx`	Plain terms for the demo.		Krishna

6. Every button (label, file, line, what it does)

Found automatically in the code (buttons and button-styled links). "calls X()" is the function the click runs; most of those are in lib/api.ts (section 9). Labels written as code (for example a price) show as their fixed text only.

Button	File : line	What it does
Back to search	`app/(client)/client/freelancers/[id]/page.tsx:35`	goes to /client/freelancers
Overview	`app/(client)/client/freelancers/[id]/page.tsx:53`	on-screen only (opens a form or switches a view)
Previous work	`app/(client)/client/freelancers/[id]/page.tsx:54`	on-screen only (opens a form or switches a view)
Reviews ()	`app/(client)/client/freelancers/[id]/page.tsx:55`	on-screen only (opens a form or switches a view)
Projects	`app/(client)/client/freelancers/[id]/page.tsx:56`	on-screen only (opens a form or switches a view)
Post a job	`app/(client)/client/freelancers/[id]/page.tsx:101`	goes to /client/jobs/new
All	`app/(client)/client/freelancers/page.tsx:59`	
Clear filters	`app/(client)/client/freelancers/page.tsx:97`	clear()
Accept	`app/(client)/client/jobs/[id]/page.tsx:110`	on-screen only (opens a form or switches a view)
Remove	`app/(client)/client/jobs/[id]/page.tsx:110`	remove()
Accept	`app/(client)/client/jobs/[id]/page.tsx:126`	on-screen only (opens a form or switches a view)
Remove	`app/(client)/client/jobs/[id]/page.tsx:127`	remove()
Open project	`app/(client)/client/jobs/[id]/page.tsx:132`	goes to /client/projects/${data.projectId}
Next	`app/(client)/client/jobs/new/page.tsx:103`	next()
Back	`app/(client)/client/jobs/new/page.tsx:121`	
Next	`app/(client)/client/jobs/new/page.tsx:122`	next()
Back	`app/(client)/client/jobs/new/page.tsx:138`	
Post a job	`app/(client)/client/jobs/page.tsx:13`	goes to /client/jobs/new
Back to the project	`app/(client)/client/projects/[id]/milestones/page.tsx:15`	goes to /client/projects/${id}
Find freelancers	`app/(client)/client/saved/page.tsx:34`	goes to /client/freelancers
Back to jobs	`app/(freelancer)/freelancer/clients/[id]/page.tsx:14`	goes to /freelancer/jobs
See all open jobs	`app/(freelancer)/freelancer/dashboard/page.tsx:102`	goes to /freelancer/jobs
Send application	`app/(freelancer)/freelancer/jobs/[id]/page.tsx:56`	submits the form

Button	File : line	What it does
Clear search	`app/(freelancer)/freelancer/jobs/page.tsx:23`	goes to /freelancer/jobs
Save	`app/(freelancer)/freelancer/profile/page.tsx:83`	submits the form
Run again	`app/demo/page.tsx:15`	reload()
Fill in	`app/login/page.tsx:97`	
Find talent	`app/start/page.tsx:69`	goes to #start
Post a project	`app/start/page.tsx:70`	goes to #start
Log in as	`app/start/page.tsx:93`	goes to /login?role=${role}
Answer	`components/call/Call.tsx:231`	accept()
Decline	`components/call/Call.tsx:232`	decline()
Voice call	`components/call/Call.tsx:238`	call()
Video call	`components/call/Call.tsx:239`	call()
End call	`components/call/Call.tsx:252`	hangUp()
Approve	`components/changes/ChangeList.tsx:49`	approveChange() → server
Reject	`components/changes/ChangeList.tsx:51`	rejectChange() → server
Reopen	`components/changes/ChangeList.tsx:57`	reopenChange() → server
Post a job	`components/dashboard/ClientDashboardView.tsx:32`	goes to /client/jobs/new
Close help	`components/help/HelpBot.tsx:72`	on-screen only (opens a form or switches a view)
Ask	`components/help/HelpBot.tsx:92`	submits the form
Search	`components/home/MarketHome.tsx:40`	submits the form
Show ${c.name}	`components/landing/CategoryCarousel.tsx:75`	go()
Clear search	`components/landing/HeroSearch.tsx:82`	
Find Talent	`components/landing/LandingNav.tsx:37`	talent()
Find Work	`components/landing/LandingNav.tsx:38`	work()
Sign up	`components/landing/LandingNav.tsx:66`	goes to /register
Log in	`components/landing/LandingNav.tsx:76`	close()
Sign up	`components/landing/LandingNav.tsx:77`	close()
Browse all freelancers	`components/landing/Sections.tsx:54`	goSearch()
View profile	`components/landing/Sections.tsx:66`	goTo()
Find Talent	`components/landing/Sections.tsx:84`	goSearch()
Start Freelancing	`components/landing/Sections.tsx:85`	goSearch()
Mark ready	`components/milestones/MilestoneList.tsx:59`	server call via lib/api.ts
Approve	`components/milestones/MilestoneList.tsx:62`	server call via lib/api.ts
Remove	`components/milestones/MilestoneSplit.tsx:53`	on-screen only (opens a form or switches a view)
Add milestone	`components/milestones/MilestoneSplit.tsx:60`	on-screen only (opens a form or switches a view)

Button	File : line	What it does
Cancel	`components/milestones/MilestoneSplit.tsx:62`	onCancel()
Previous review	`components/reviews/ReviewCarousel.tsx:96`	go()
Show review ${i + 1} of ${n}	`components/reviews/ReviewCarousel.tsx:99`	go()
Next review	`components/reviews/ReviewCarousel.tsx:102`	go()
Open your account menu	`components/shell/AppShell.tsx:187`	toggle()
Log out	`components/shell/AppShell.tsx:204`	
Close	`components/shell/AppShell.tsx:212`	on-screen only (opens a form or switches a view)
Approve SOW	`components/sow/SowReview.tsx:105`	approve()
Send back	`components/sow/SowReview.tsx:106`	on-screen only (opens a form or switches a view)
Confirm hours and sign	`components/sow/SowReview.tsx:118`	sends "confirm" to the server
Send the hours back	`components/sow/SowReview.tsx:119`	on-screen only (opens a form or switches a view)
Send back	`components/sow/SowReview.tsx:131`	sends "send_back" to the server
Cancel	`components/sow/SowReview.tsx:132`	on-screen only (opens a form or switches a view)
Download the sample SOW	`components/sow/SowReview.tsx:149`	goes to /sample-sow.txt
Cancel	`components/tasks/TaskRequests.tsx:74`	onCancel()
Quote a price	`components/tasks/TaskRequests.tsx:142`	on-screen only (opens a form or switches a view)
Already included	`components/tasks/TaskRequests.tsx:143`	sends "included" to the server
Our defect: fix free	`components/tasks/TaskRequests.tsx:144`	on-screen only (opens a form or switches a view)
Ask for details	`components/tasks/TaskRequests.tsx:145`	on-screen only (opens a form or switches a view)
Decline	`components/tasks/TaskRequests.tsx:146`	on-screen only (opens a form or switches a view)
Add details	`components/tasks/TaskRequests.tsx:152`	on-screen only (opens a form or switches a view)
Withdraw request	`components/tasks/TaskRequests.tsx:153`	on-screen only (opens a form or switches a view)
Accept	`components/tasks/TaskRequests.tsx:159`	sends "accept" to the server
Counter offer	`components/tasks/TaskRequests.tsx:160`	on-screen only (opens a form or switches a view)
Decline	`components/tasks/TaskRequests.tsx:161`	on-screen only (opens a form or switches a view)
Confirm: fix at no charge	`components/tasks/TaskRequests.tsx:172`	sends "defect" to the server

Button	File : line	What it does
Cancel	`components/tasks/TaskRequests.tsx:172`	on-screen only (opens a form or switches a view)
Cancel	`components/tasks/TaskRequests.tsx:185`	on-screen only (opens a form or switches a view)
Send to the freelancer	`components/tasks/TaskRequests.tsx:238`	ask()
See profile	`components/ui/FreelancerCard.tsx:46`	
Post check-in	`components/work/CheckIns.tsx:73`	send()
Send back	`components/work/WorkBoard.tsx:52`	moveWork() → server
Cancel	`components/work/WorkBoard.tsx:53`	on-screen only (opens a form or switches a view)
Start	`components/work/WorkBoard.tsx:59`	moveWork() → server
Mark done	`components/work/WorkBoard.tsx:60`	moveWork() → server
Approve	`components/work/WorkBoard.tsx:63`	moveWork() → server
Send back	`components/work/WorkBoard.tsx:64`	on-screen only (opens a form or switches a view)

7. Backend: API routes, services and rules

The backend is the same Next.js project: **API routes** in `app/api` (thin: read the request, call one service function), the **service layer** in `lib/server` (who may do what, wiring), and the **rules engine** in `lib/engine` (pure functions, unit tested). On Vercel each route runs as a serverless function.

Server files (lib/server)

File	What it does
`lib/server/auth.ts`	Accounts: register, log in (password), and sign in with Google. Passwords are checked as salted hashes, sessions are signed cookies, repeated wrong passwords lock the email for a while. Email verification and password reset need email (Brevo, planned): until then accounts start with emailVerified false and the Google ones with true.
`lib/server/context.ts`	What a request carries: which side it acts as (x-demo-role), the signed-in user from the session cookie, and any cookies the handler wants set on the response. run() fills it in and actor() reads it.
`lib/server/errors.ts`	
`lib/server/google.ts`	Sign in with Google (OAuth 2.0 authorisation code flow). It is switched on by GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET. Set the redirect URI in the Google Cloud console to <your site>/api/auth/google/callback. The endpoints can be replaced with GOOGLE_AUTH_URL, GOOGLE_TOKEN_URL and GOOGLE_USERINFO_URL, which is how it is tested without Google.
`lib/server/http.ts`	
`lib/server/library.ts`	The SOW library: typical rate and hours per job profile. Data is a snapshot in lib/data/sowLibrary.json (see its _note). Replace it as more SOWs are written.

File	What it does
`lib/server/persist.ts`	Keeps the app's data in Supabase (Postgres) when SUPABASE_URL (or NEXT_PUBLIC_SUPABASE_URL) and SUPABASE_SERVICE_ROLE_KEY are set; otherwise everything stays in memory, as in development. How it works (a deliberate hackathon shortcut, not a normalised schema): - The whole app state is ONE versioned JSON row in the table app_state. Each request loads its own private copy (db() returns that copy), and if the request changed anything it writes the copy back with "update ... where version = <the version I loaded>". If someone else wrote in between, the write is refused and the whole request is run again on the fresh data, so no update is lost. - Call signalling (very frequent, short-lived) lives in its own table, call_signals, so calls do not rewrite the state. - Talks to Supabase's REST interface with fetch, so no extra package is needed. The service key stays on the server.
`lib/server/service.ts`	Business operations with role checks. Money rules live in lib/engine (plain JS, tested). This file wires people, jobs and projects to it.
`lib/server/session.ts`	Signed session cookie. The secret comes from SESSION_SECRET; without one a random secret is made when the server starts, so everyone is signed out after a restart (fine for the demo, set SESSION_SECRET when deployed).
`lib/server/store.ts`	The app's data and its demo seed. db() is the ONLY way the rest of the code reaches data. In development it is one in-memory object. When Supabase is configured (see persist.ts) each request gets its own copy, loaded from the database and saved back after a change.

Rules engine (lib/engine)

File	What it does	Unit tests
`lib/engine/aiguard.js`	AI second check for the contact guard. The rule-based guard (guard.js) always runs first; this asks a language model whether a message or a spoken line is trying to share contact details or move the conversation off the platform in a way the rules cannot see ("my number is the year India got independence, then the Hyderabad pin code"). It is optional: with no key, or if the model is slow, down or answers badly, it returns null and the rules decide alone. Free option: Google Gemini (key from aistudio.google.com). Also supported: Anthropic Claude. Plain JavaScript, tested with a stand-in fetch, so no network is needed to test it. Never put a key in the repo: it is read from the environment.	9
`lib/engine/aihelp.js`	AI answers for the Help assistant. A language model (the same free Gemini key as the contact guard) writes the reply, but it is only allowed to use the written guide in helpbot.js, so it explains how THIS site works instead of making things up. With no key, or if the model is slow or down, it returns null and the keyword answer from helpbot.js is used. Plain JavaScript, tested with a stand-in fetch.	6
`lib/engine/authcore.js`	Password hashing, signed session cookies, signed one-time state, and a log-in attempt limiter. Uses only node:crypto. Passwords are never stored: only a salted scrypt hash. Sessions are signed (HMAC-SHA256) so they cannot be forged, and they expire. Plain JavaScript, tested.	

File	What it does	Unit tests
`lib/engine/demo.js`	Scripted demo of the three case outcome checks, in printed order. Runs on its own throwaway desk so it never touches the user's working state.	2
`lib/engine/exporter.js`	CSV helpers for the saved record. Values starting with = + - @ are prefixed with a quote so a spreadsheet will not run them as formulas.	
`lib/engine/guard.js`	Contact guard: finds phone numbers, email addresses, social handles, links and similar personal details in text, including when they are written or spoken in words ("eight one two two nine ...", "name at gmail dot com"). It is used on chat messages (the details are hidden before saving) and on live call captions (the call is ended). It is a rule-based guard that works offline and is fully tested; a language model could be added later as a second opinion, but nothing here calls a model. Plain JavaScript, no dependencies.	14
`lib/engine/helpbot.js`	Help assistant: answers common questions from clients and freelancers using this site's own guide. It is rule-based (keyword matching against a written guide), works offline, and never makes things up: if nothing matches well it says so and offers topics. It is NOT a language model. Plain JavaScript, tested.	6
`lib/engine/negotiation.js`	Negotiation: the client and the freelancer go back and forth on hours, price and date until one side accepts the other side's latest offer, or someone declines. One mechanism, used when hiring, on SOW rows and on every new task (docs/SCOPE_FLOW.md, decision D-76). Nothing is committed until both sides agree. No dependencies. Rule from the case data: null means "unknown", never zero, so an offer with unknown hours has no price and cannot be accepted.	13
`lib/engine/progress.js`	Work progress and daily check-ins (docs/SCOPE_FLOW.md section 2, decision D-76). No dependencies. - A task moves: to do -> in progress -> done (the freelancer) -> approved (the client). The client can send a done task back to in progress, with a reason. - Progress = approved hours / all known hours. Tasks with unknown hours are counted apart, never as zero. - Check-ins: the freelancer posts one a day (India time). The client is warned after 2 days without one.	5
`lib/engine/scopeDesk.js`	Scope Change Desk rules engine. No dependencies. The UI only calls this module. Rule from the case data: null means "unknown", never zero.	17
`lib/engine/sow.js`	SOW reader and comparison. Plain JavaScript so it is tested with the rest of the engine. Understands text or markdown with headings such as "Deliverables", "Included deliverables", "Included", "Excluded (needs a change order)", followed by bullet lines. A person always reviews the result; this is a starting point, not a promise of accuracy.	5
`lib/engine/taskrules.js`	Suggests what a client's plain-English request is, compared with the signed scope: already included, excluded in the SOW, new work, a fix of something that should already work, or too unclear to price. It is only a suggestion: the freelancer decides (docs/SCOPE_FLOW.md, step 3). Rule-based, offline, no dependencies.	4

File	What it does	Unit tests
`lib/engine/validate.js`	Validation for sign-up and log-in. The SAME rules run in the browser (to show messages as you type) and on the server (which is the one that counts). Each function returns an error message, or '' when the value is fine. Plain JavaScript, no dependencies, tested.	

Errors

Every failure becomes JSON `{ error, type, fields? }` with a status: 400 bad input (with a message per form field), 401/403 not signed in or not allowed, 404 not found, 409 conflict (already done, not your turn, over capacity), 503 busy (retry). Nothing crashes the page.

8. Every API route (URL, method, file, server function)

URL	Method	File	Calls	Note
`/api/ai-status`	GET	`app/api/ai-status/route.ts`	status (lib/engine/aiguard)	Whether the AI second check is on (a key is set) and which provider. Never returns the key.
`/api/applications/:id/accept`	POST	`app/api/applications/[id]/accept/route.ts`	acceptApplication (lib/server/service)	
`/api/applications/:id/reject`	POST	`app/api/applications/[id]/reject/route.ts`	rejectApplication (lib/server/service)	
`/api/auth/add-role`	POST	`app/api/auth/add-role/route.ts`	addRole (lib/server/auth)	Adds the other side (client or freelancer) to the signed-in account.
`/api/auth/config`	GET	`app/api/auth/config/route.ts`	googleConfig (lib/server/google)	What the sign-in pages can offer. Never returns a secret.
`/api/auth/google/callback`	GET	`app/api/auth/google/callback/route.ts`	readCookie (lib/engine/authcore), googleSignIn (lib/server/auth), fetchProfile (lib/server/google), googleConfig (lib/server/google), requestRole (lib/server/context), OAUTH_COOKIE (lib/server/session), clearOauthCookie (lib/server/session), readOauth (lib/server/session), withState (lib/server/persist)	Step 2: Google sends the person back with a one-time code. Check the state, get their verified profile, sign them in.
`/api/auth/google/start`	GET	`app/api/auth/google/start/route.ts`	randomToken (lib/engine/authcore), authorizeUrl (lib/server/google), googleConfig (lib/server/google), oauthCookie (lib/server/session)	Step 1: send the person to Google. The state is kept in a signed cookie so the way back can be trusted.
`/api/auth/logout`	POST	`app/api/auth/logout/route.ts`	logoutAccount (lib/server/auth)	
`/api/auth/register`	POST	`app/api/auth/register/route.ts`	registerAccount (lib/server/auth)	

URL	Method	File	Calls	Note
`/api/client-dashboard`	GET	`app/api/client-dashboard/route.ts`	clientDashboard (lib/server/service)	
`/api/clients/:id`	GET	`app/api/clients/[id]/route.ts`	getClientProfile (lib/server/service)	
`/api/dashboard`	GET	`app/api/dashboard/route.ts`	freelancerDashboard (lib/server/service)	
`/api/demo`	GET	`app/api/demo/route.ts`	demo (lib/server/service)	
`/api/freelancers/:id`	GET	`app/api/freelancers/[id]/route.ts`	getFreelancerProfile (lib/server/service)	
`/api/freelancers`	GET	`app/api/freelancers/route.ts`	searchFreelancers (lib/server/service)	
`/api/help`	POST	`app/api/help/route.ts`	answer (lib/engine/helpbot), ask (lib/engine/aihelp)	The Help assistant. The keyword answer is always worked out; if an AI key is set, a model writes the reply from the site guide instead (and only from the guide). If the model is off, slow or busy, the keyword answer is returned. A small limit keeps the free AI quota from being used up: 20 AI answers a minute for the whole site.
`/api/jobs/:id/applications`	POST	`app/api/jobs/[id]/applications/route.ts`	applyToJob (lib/server/service)	
`/api/jobs/:id`	GET	`app/api/jobs/[id]/route.ts`	getJob (lib/server/service)	
`/api/jobs`	GET, POST	`app/api/jobs/route.ts`	createJob (lib/server/service), listJobs (lib/server/service)	
`/api/library`	GET	`app/api/library/route.ts`	library (lib/server/library)	
`/api/login`	POST	`app/api/login/route.ts`	loginAccount (lib/server/auth)	

URL	Method	File	Calls	Note
`/api/me`	GET, PUT	`app/api/me/route.ts`	updateMe (lib/server/service)	
`/api/projects/:id/call/caption`	POST	`app/api/projects/[id]/call/caption/route.ts`	reportCaption (lib/server/service)	The AI check can take a few seconds, so allow the function more time on the host.
`/api/projects/:id/call`	POST	`app/api/projects/[id]/call/route.ts`	startCall (lib/server/service)	
`/api/projects/:id/call/signal`	POST	`app/api/projects/[id]/call/signal/route.ts`	sendCallSignal (lib/server/service)	
`/api/projects/:id/call/signals`	GET	`app/api/projects/[id]/call/signals/route.ts`	listCallSignals (lib/server/service)	
`/api/projects/:id/changes/:changeId/:action`	POST	`app/api/projects/[id]/changes/[changeId]/[action]/route.ts`	approveChange (lib/server/service), rejectChange (lib/server/service), reopenChange (lib/server/service)	
`/api/projects/:id/checkins`	POST	`app/api/projects/[id]/checkins/route.ts`	addCheckin (lib/server/service)	The freelancer's daily check-in: { did, hours, blockers?, moved?: [{ key, to }] }. One a day (India time).
`/api/projects/:id/messages`	GET, POST	`app/api/projects/[id]/messages/route.ts`	listMessages (lib/server/service), sendMessage (lib/server/service)	The AI check can take a few seconds, so allow the function more time on the host.
`/api/projects/:id/milestones/:milestoneId/:action`	POST	`app/api/projects/[id]/milestones/[milestoneId]/[action]/route.ts`	approveMilestone (lib/server/service), markMilestoneReady (lib/server/service)	

URL	Method	File	Calls	Note
`/api/projects/:id`	GET	`app/api/projects/[id]/route.ts`	getProject (lib/server/service)	
`/api/projects/:id/sow/:docId`	POST	`app/api/projects/[id]/sow/[docId]/route.ts`	reviewSow (lib/server/service)	Sign-off of the first SOW. Freelancer: { action: approve, rows? } or { action: send_back, note }. Client: { action: confirm } or { action: send_back, note }. Returns the whole project.
`/api/projects/:id/sow`	POST	`app/api/projects/[id]/sow/route.ts`	uploadSow (lib/server/service)	
`/api/projects/:id/tasks/:taskId`	POST	`app/api/projects/[id]/tasks/[taskId]/route.ts`	actOnTask (lib/server/service)	One step on a task request: { action: included | defect | quote | ask | details | decline | counter | accept, ... }. Returns the whole project, so the page can redraw the totals too.
`/api/projects/:id/tasks`	GET, POST	`app/api/projects/[id]/tasks/route.ts`	createTask (lib/server/service), listTasks (lib/server/service)	New tasks asked for in plain English: list them, or (client) ask for one.
`/api/projects/:id/work/:key`	POST	`app/api/projects/[id]/work/[key]/route.ts`	actOnWork (lib/server/service)	One task on the task list: { action: 'move', to, note? } or { action: 'milestone', milestoneId }. Returns the project. The key looks like sow:S1:0 or task:T12 (sent URL-encoded).
`/api/projects`	GET	`app/api/projects/route.ts`	listProjects (lib/server/service)	
`/api/reset`	POST	`app/api/reset/route.ts`	clearSignals (lib/server/persist), persistent (lib/server/persist), resetDb (lib/server/store)	Puts the demo data back. For demos and tests only: in production it is OFF unless ALLOW_RESET=true is set, because anyone who found it could wipe the live data.
`/api/saved/:id`	PUT	`app/api/saved/[id]/route.ts`	setSaved (lib/server/service)	PUT { saved: true | false } saves or removes a freelancer from the client's shortlist.
`/api/saved`	GET	`app/api/saved/route.ts`	listSaved (lib/server/service)	
`/api/stats`	GET	`app/api/stats/route.ts`	siteStats (lib/server/service)	Public: real counts from our own data, shown on the landing page. No sign-in needed.
`/api/users`	GET	`app/api/users/route.ts`	listUsers (lib/server/service)	

URL	Method	File	Calls	Note
`/api/version`	GET	`app/api/version/route.ts`		Which commit this server was built from, so a deploy can be checked from outside. Vercel sets the variables; locally they are empty. Nothing secret is returned.

Full request/response details: `docs/API_CONTRACT.md`.

9. Browser API functions (lib/api.ts)

Pages and components call these; each sends one HTTPS request to the route on the right and returns the JSON (or throws an error whose message is safe to show).

Function	Method	Goes to	Returns
`getFreelancer`	GET	`/api/freelancers/:id`	`FreelancerProfile`
`getClient`	GET	`/api/clients/:id`	`ClientProfile`
`setSaved`	PUT	`/api/saved/:id`	`{ saved: boolean }`
`listJobs`	GET	`/api/jobs?…`	`Job[]`
`getJob`	GET	`/api/jobs/:id`	`JobDetail`
`applyToJob`	POST	`/api/jobs/:jobId/applications`	`Application`
`acceptApplication`	POST	`/api/applications/:applicationId/accept`	`{ projectId: string }`
`rejectApplication`	POST	`/api/applications/:applicationId/reject`	`{ id: string; status: 'rejected' }`
`getProject`	GET	`/api/projects/:id`	`ProjectDetail`
`listMessages`	GET	`/api/projects/:projectId/messages`	`Message[]`
`sendMessage`	POST	`/api/projects/:projectId/messages`	`Message`
`markMilestoneReady`	POST	`/api/projects/:projectId/milestones/:milestoneId/ready`	`ProjectDetail`
`approveMilestone`	POST	`/api/projects/:projectId/milestones/:milestoneId/approve`	`ProjectDetail`
`uploadSow`	POST	`/api/projects/:projectId/sow`	`UploadSowResult`
`moveWork`	POST	`/api/projects/:projectId/work/:key`	`ProjectDetail`
`setWorkMilestone`	POST	`/api/projects/:projectId/work/:key`	`ProjectDetail`
`addCheckin`	POST	`/api/projects/:projectId/checkins`	`ProjectDetail`
`reviewSow`	POST	`/api/projects/:projectId/sow/:docId`	`ProjectDetail`
`approveChange`	POST	`/api/projects/:projectId/changes/:changeId/approve`	`ProjectDetail`
`rejectChange`	POST	`/api/projects/:projectId/changes/:changeId/reject`	`ProjectDetail`
`reopenChange`	POST	`/api/projects/:projectId/changes/:changeId/reopen`	`ProjectDetail`
`listTasks`	GET	`/api/projects/:projectId/tasks`	`TaskRequest[]`
`askForTask`	POST	`/api/projects/:projectId/tasks`	`{ task: TaskRequest; created: boolean; project: ProjectDetail }`
`actOnTask`	POST	`/api/projects/:projectId/tasks/:taskId`	`ProjectDetail`

Function	Method	Goes to	Returns
`startCall`	POST	`/api/projects/:projectId/call`	`CallSignal`
`pollCallSignals`	GET	`/api/projects/:projectId/call/signals?…`	`CallSignal[]`
`sendCallSignal`	POST	`/api/projects/:projectId/call/signal`	`CallSignal`
`sendCaption`	POST	`/api/projects/:projectId/call/caption`	`CaptionResult`
`listDemoUsers`	GET	`/api/users`	`User[]`
`getMe`	GET	`/api/me`	`User`
`register`	POST	`/api/auth/register`	`User`
`addRole`	POST	`/api/auth/add-role`	`User`
`logout`	POST	`/api/auth/logout`	`{ ok: true }`
`getAuthConfig`	GET	`/api/auth/config`	`{ google: boolean }`
`login`	POST	`/api/login`	`User`
`updateMe`	PUT	`/api/me`	`User`
`listSaved`	GET	`/api/saved`	`Freelancer[]`
`createJob`	POST	`/api/jobs`	`Job`
`getLibrary`	GET	`/api/library`	`LibraryResponse`
`listProjects`	GET	`/api/projects`	`Project[]`
`getFreelancerDashboard`	GET	`/api/dashboard`	`FreelancerDashboard`
`getClientDashboard`	GET	`/api/client-dashboard`	`ClientDashboard`
`getSiteStats`	GET	`/api/stats`	`SiteStats`
`getAiStatus`	GET	`/api/ai-status`	`{ enabled: boolean; provider: string | null }`
`runCaseChecks`	GET	`/api/demo`	`DemoResult`
`resetData`	POST	`/api/reset`	`{ ok: true }`

10. Database: Supabase, tables and the data schema

Where: a Supabase project (hosted Postgres). **Tables:** two, created by `supabase/schema.sql`:

```
-- Crewmate: the two tables the app needs. Run this once in Supabase: SQL Editor, paste, Run. Safe to run
again.
--
-- app_state    one row holding the whole app's data as JSON, with a version number. The app writes it with
--              "update ... where version = n", so two people saving at once never overwrite each other.
-- call_signals the short-lived messages that set up voice and video calls (ring, offer, answer, captions,
hang up).
--
-- Row level security is switched ON with no policies, so nobody can read or write these tables with the
public
-- (anon) key. Only the server, using the service key, can. Never put the service key in the browser or in
git.

create table if not exists public.app_state (
  id         text primary key,
  version    bigint not null default 0,
  data       jsonb,
  updated_at timestamptz not null default now()
);

insert into public.app_state (id) values ('main') on conflict (id) do nothing;

create table if not exists public.call_signals (
  seq        bigserial primary key,
  project_id text not null,
  from_user  text not null,
  to_user    text not null,
  type       text not null,
  payload    text not null default '',
  at         timestamptz not null default now()
);

create index if not exists call_signals_lookup on public.call_signals (project_id, to_user, seq);
create index if not exists call_signals_age on public.call_signals (at);

alter table public.app_state enable row level security;
alter table public.call_signals enable row level security;
```

> **Design (a deliberate hackathon choice):** the whole app state is ONE versioned JSON row (`app_state`, id `main`). Each request loads its own copy, works on it with the tested rules, and saves it back only if it changed, with "update where version = n" and a retry if someone saved first (tested with 14 parallel writers and a restart). Call signalling changes many times a second, so it has its own table. **Trade-off:** simple and safe for a demo and pilot, but one row will not scale to many users; the next step is a normalised schema (one table per entity below). Older rows simply lack newer fields and read as empty, so no migrations were needed.

What is inside the JSON (the logical schema: lib/server/store.ts)

Field	Type	Meaning
`users`	`User[]`	

Field	Type	Meaning
`jobs`	`Job[]`	
`applications`	`Application[]`	
`projects`	`Project[]`	
`sow`	`SowDocument[]`	
`messages`	`Message[]`	
`signals`	`CallSignal[]`	call signalling, in memory
`credentials`	`Map<string, string>`	user id -> salted password hash (never sent to the browser)
`googleIds`	`Map<string, string>`	Google account id -> user id
`limiter`	`Limiter`	log-in attempt limiter
`milestones`	`Milestone[]`	
`saved`	`Map<string, Set<string>>`	client id -> saved freelancer ids
`desks`	`Map<string, ScopeDesk>`	
`labels`	`Map<string, Record<string, CompareLabel>>`	
`hashes`	`Map<string, string>`	projectId:sha256(text) -> sow document id, to ignore repeat uploads
`tasks`	`TaskRequest[]`	new tasks asked for in plain English
`deals`	`Map<string, Negotiations>`	project id -> its negotiations (quotes on new tasks)
`work`	`WorkState[]`	status and milestone of each task (the task itself comes from the SOW or a request)
`checkins`	`CheckIn[]`	daily check-ins
`seq`	`number`	

User

Field	Type	Meaning
`id`	`string`	
`name`	`string`	
`role`	`Role`	
`skills`	`string[]`	
`emailVerified?`	`boolean`	false until the verification email (Brevo, planned) is confirmed; Google accounts start verified
`email?`	`string`	what the person entered when they registered (demo accounts only for now)
`roles?`	`Role[]`	an account that can use both sides lists both; `role` is the side it is acting as right now
`demo?`	`boolean`	shown on the start page as a one-click demo login
`headline?`	`string`	

Field	Type	Meaning
`overview?`	`string`	
`location?`	`string`	
`ratePerHour?`	`number`	
`category?`	`string`	matches a library category
`availability?`	`'available' | 'busy'`	
`rating?`	`number`	0 to 5, sample data
`jobsDone?`	`number`	sample data

Job

Field	Type	Meaning
`id`	`string`	
`clientId`	`string`	
`title`	`string`	
`description`	`string`	
`budget`	`number`	
`deadline`	`string`	ISO date, e.g. 2026-11-10
`profileId?`	`string`	library profile the job started from (sets the project rate and hours when accepted)
`category?`	`string`	
`status`	`JobStatus`	
`createdAt`	`string`	

Application

Field	Type	Meaning
`id`	`string`	
`jobId`	`string`	
`jobTitle?`	`string`	filled in by the dashboard endpoint
`freelancerSkills?`	`string[]`	filled in by the job endpoint, for comparing proposals
`freelancerId`	`string`	
`freelancerName`	`string`	
`pitch`	`string`	
`proposedPrice`	`number`	
`status`	`ApplicationStatus`	
`createdAt`	`string`	

Project

Field	Type	Meaning
`id`	`string`	
`jobId`	`string`	
`jobTitle`	`string`	
`clientId`	`string`	
`freelancerId`	`string`	
`ratePerHour`	`number`	
`availableHours`	`number`	
`currency`	`'INR'`	
`createdAt`	`string`	

Milestone

Field	Type	Meaning
`id`	`string`	
`projectId`	`string`	
`title`	`string`	
`amount`	`number`	whole rupees, before tax
`status`	`MilestoneStatus`	
`readyAt`	`string | null`	freelancer marked it ready for review
`approvedAt`	`string | null`	
`approvedBy`	`string | null`	

SowDocument

Field	Type	Meaning
`id`	`string`	
`projectId`	`string`	
`version`	`number`	
`kind`	`SowKind`	
`filename`	`string`	
`uploadedBy`	`string`	
`uploadedAt`	`string`	
`items`	`SowItem[]`	
`status?`	`SowStatus`	
`ratePerHour?`	`number | null`	a "Rate:" line read from the file, applied when signed
`signedAt?`	`string`	

SowItem

Field	Type	Meaning
`section`	`SowSection`	
`text`	`string`	
`hours`	`number | null`	

Message

Field	Type	Meaning
`id`	`string`	
`projectId`	`string`	
`senderId`	`string`	
`senderName`	`string`	
`body`	`string`	
`sentAt`	`string`	
`redacted?`	`boolean`	true when the server hid contact details in the body
`hiddenKinds?`	`string[]`	what was hidden, for example 'phone number (said in words)'
`warning?`	`string`	set when the text asks to move off the platform (the text is kept)

TaskRequest

Field	Type	Meaning
`id`	`string`	
`projectId`	`string`	
`text`	`string`	
`details`	`string[]`	the client's answers when asked for details
`urgent`	`boolean`	
`askedAt`	`string`	
`status`	`TaskStatus`	
`negotiationId`	`string | null`	
`bookedHours`	`number | null`	hours booked for an own-defect fix (no charge)
`events`	`TaskEvent[]`	
`negotiation?`	`import('./engine/negotiation').NegotiationView | null`	filled in when read

WorkState

Field	Type	Meaning
`key`	`string`	
`projectId`	`string`	

Field	Type	Meaning
`status`	`WorkStatus`	
`milestoneId`	`string | null`	
`events`	`WorkItem['events']`	

CheckIn

Field	Type	Meaning
`id`	`string`	
`projectId`	`string`	
`day`	`string`	YYYY-MM-DD, India time
`at`	`string`	
`did`	`string`	
`hours`	`number`	
`blockers`	`string`	

CallSignal

Field	Type	Meaning
`id`	`string`	
`seq`	`number`	grows with every signal; the poller asks for signals newer than the last one it saw
`projectId`	`string`	
`fromUserId`	`string`	
`toUserId`	`string`	only the other project member receives this signal
`type`	`CallSignalType`	
`payload`	`string`	JSON for ring (mode), offer/answer (SDP), ice (candidate), hangup (reason); text for caption
`at`	`string`	

11. How the frontend is linked to Supabase

> **It is not linked directly.** The browser never holds a Supabase key and never calls Supabase. The chain is: page → `lib/api.ts` → our API route on Vercel → `lib/server/persist.ts` → Supabase REST (`https://<project>.supabase.co/rest/v1/app_state`) with the secret key from the server's environment variables.

- `persist.ts` reads `NEXT_PUBLIC_SUPABASE_URL` (or SUPABASE_URL) and `SUPABASE_SERVICE_ROLE_KEY`. With both set it uses Supabase; otherwise memory. A publishable (public) key is refused with a warning.
- Requests use `fetch` with the `apikey` header (and a Bearer token for older JWT keys), an 8-second timeout, and no extra npm package.
- Because RLS is on with no policies, even someone who found the public URL cannot read the tables.
- Check it works: `npm run db:check` (against a local pretend Supabase) or the Table Editor in Supabase.

12. Login, sessions and Google sign-in

- **Register** (`/register` → `POST /api/auth/register`): validated on the server too (name, email, password strength, common passwords refused, terms). Password stored as a salted scrypt hash.
- **Log in** (`/login` → `POST /api/login`): sets the `cd_session` cookie (HTTP-only, Secure in production, SameSite). 5 wrong passwords lock the email for 10 minutes (saved in the database, so it holds across servers). A wrong password and an unknown email give the same message.
- **One account, both sides:** an account can be client, freelancer, or both; the avatar menu switches side or creates the other side (`POST /api/auth/add-role`).
- **Google:** `/api/auth/google/start` → Google → `/api/auth/google/callback` (OAuth code flow, state checked) → `/auth/done`. Needs GOOGLE_CLIENT_ID/SECRET and the live callback URL in Google Cloud.
- **Demo accounts** (`client@demo.test`, `freelancer@demo.test`, `both@demo.test`, password `demo1234`): may also be named with the `x-demo-user` header so tests and judges can act without passwords; real accounts cannot be impersonated this way.
- **Not yet:** email verification and password reset (Brevo email planned).

13. AI features and their fallbacks

- **Contact guard** (`lib/engine/guard.js`): hides phone numbers, emails, links, social handles, also when written in words ("nine eight seven ...") or sent one digit per message. Rules first, always.
- **AI second check** (`lib/engine/aiguard.js`, Gemini `gemini-flash-lite-latest`): only if the rules found nothing and GEMINI_API_KEY is set; if the model is slow or down, the rules decide. Labelled "hidden by the AI check".
- **Calls:** live captions are checked the same way; saying contact details ends the call for both people and leaves a note in the chat.
- **Help assistant** (`lib/engine/aihelp.js` + `helpbot.js`): answers how-to questions; without a key, a built-in rule-based help answers.
- **Plain-English request labels** (`lib/engine/taskrules.js`): rule-based and shown as "suggested by rules, the freelancer decides". AI labelling is not added yet.

14. Business rules and the three judged checks

Check (in printed order)	How the app guarantees it	Where it is tested
1. Six extra hours at ₹1000/hr = ₹6000 change estimate before taxes	Price = hours × rate (× 1.5 rush, labelled an assumption), always "before tax". In change orders (scopeDesk.js) and in negotiated requests (negotiation.js).	scopeDesk.test.js, negotiation.test.js, smoke test (agreed at ₹6000 after counter offers), /demo page
2. A rejected change does not silently become committed work	Only approved change orders and agreed negotiations count in committed hours/value. Declined ones are closed and cannot be accepted later. Only the side that did not make the latest offer can accept it.	Same tests + smoke
3. Repeated import of the same request does not double-charge or double-book	Requests keyed by id + SHA-256 of content; the same SOW text or the same plain-English request returns the existing one.	Same tests + smoke

- **Unknown is not zero:** hours not given stay `null`; such items show "no price yet" and cannot be accepted.
- **Capacity gate:** nothing can be approved or agreed beyond the hours left in the project (409 with a clear message).
- **Edge cases:** unclear request → no price until details; urgent → rush fee as an assumption; own defect → no charge, hours still booked; conflicting approvals → turn-based acceptance.
- **Milestones:** the price is split into milestones at hiring (must add up exactly); record only, no money moves.

15. Deployment (Vercel + Supabase) and environment variables

1. Code is on GitHub (`Krishnass05/fynd`). Vercel project **crewmate** builds the `krishna` branch (Production Branch Tracking) on every push, in about 1.5 to 3.5 minutes; `/api/version` shows the live commit.
2. Supabase project: `supabase/schema.sql` run once in the SQL Editor. The first request fills the demo data.
3. Google Cloud OAuth client with the live redirect URL `https://crewmate-sable.vercel.app/api/auth/google/callback`.
4. On the live site `/api/reset` is switched off (unless ALLOW_RESET=true, which is not set).

Variable	Purpose	Secret?
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL	No
`SUPABASE_SERVICE_ROLE_KEY`	Supabase secret key (server only)	Yes: only in .env.local and Vercel settings
`STORE`	memory forces in-memory data	No
`ALLOW_RESET`	allow wiping data in production (keep off)	No
`GEMINI_API_KEY`	Gemini key for the AI checks and help	Yes: only in .env.local and Vercel settings
`AI_GUARD_PROVIDER`	gemini, anthropic or off	No
`AI_GUARD_MODEL`	model name override	No
`GOOGLE_CLIENT_ID`	Google sign-in client	No
`GOOGLE_CLIENT_SECRET`	Google sign-in secret	Yes: only in .env.local and Vercel settings
`APP_URL`	live address, for the Google redirect	No

Variable	Purpose	Secret?
`SESSION_SECRET`	signs the session cookie	Yes: only in .env.local and Vercel settings

Locally: copy `.env.example` to `.env.local` (git-ignored), `npm install`, `npm run dev`, open http://localhost:3000. Step-by-step: `docs/DEPLOY.md`.

16. Testing

- `npm test`: 92 unit tests over the rules engine (money, negotiation, SOW reading, contact guard, AI fallbacks, auth, progress and check-ins).
- `npm run smoke`: the full journey against a running server with in-memory data (login, hiring, milestones, chat guard, calls, SOW sign-off, amendments and the three checks, plain-English requests and negotiation, tasks, progress, check-ins, accounts and lockout). It resets the data first, so it never runs against the live database.
- `npx tsc --noEmit` type check and `npm run build` production build.
- Helper checks: `npm run db:check`, `npm run google:check`, `npm run ai:check`.

17. What has been built (feature list)

- Landing page (Crewmate brand, logo, Hire/Work toggle, categories, how it works) and start page.
- Register / login with validation, sessions, lockout, Google sign-in; one account with both sides and a side switcher.
- Client: home with search and reviews, find freelancers (search, filters, categories from the Job counts list), profiles with past work and reviews, saved list, post a job (optionally from the SOW library of 28 job profiles), compare applicants, hire with a milestone split, dashboard with attention items and activity.
- Freelancer: home, find jobs, apply, profile editing, client profiles with company details, dashboard.
- Project workspace (both sides): SOW sign-off table (client uploads, freelancer reviews, client confirms hours) with a format guide and sample SOW; plain-English new tasks with rule labels and negotiation; task list with progress by approved hours; daily check-ins with a 2-day warning; milestones; change orders from amended SOWs; chat with the contact guard; voice and video calls with live captions and the guard; help assistant.
- Case checks page (`/demo`) runs the three judged checks in printed order.
- Supabase storage, Vercel deployment, favicon and logo.

18. Known limits and honest answers

- One JSON row in the database: fine for a demo/pilot, will not scale; next step is normalised tables.
- No email yet (verification, password reset, notifications); the other side sees changes on reload or polling.
- SOW files: .txt and .md only (no PDF reading yet); no AI reading of messy SOWs; no per-row query threads or due-date columns yet.
- Negotiation at hiring (step 5) not built yet: the client hires at the applicant's price.
- Calls need a TURN server on strict networks; tested between browsers on the same machine.
- All data is synthetic; no live revenue or production readiness is claimed.

19. Team ownership

```
Editing only your own folders is how we avoid merge conflicts. GitHub also asks the owner to review changes
to their folders (see `.github/CODEOWNERS`).

| Person | GitHub | Folders and files only they edit |
| --- | --- | --- |
| Krishna | Krishnass05 | `app/api/**`, `lib/**` (engine, server, types, api, session, hooks, nav),
`supabase/**`, `app/(freelancer)/**`, `app/page.tsx`, `app/demo/**`, `components/sow/**`,
`components/milestones/**`, `components/tasks/**`, `components/work/**`,
`app/(client)/client/freelancers/**`, `components/fx/**` (React Bits animations), `docs/DESIGN_SYSTEM.md`,
`decision.md`, `flow.md`, `scripts/**`, `fixture.json`, `docs/API_CONTRACT.md` |
| Sanjana | vincezo-ks | `components/ui/**`, `components/shell/**`, `app/globals.css`,
`app/(client)/layout.tsx`, `app/(client)/client/page.tsx`, `app/(client)/client/jobs/**`, her `frontend/`
design folder |
| Harini | chinthakuntaharini | `components/chat/**`, `components/changes/**`,
`app/(client)/client/projects/**` |
| Swapnil | swapnil2648r-droid | `docs/sow/**` (his own files), demo script and business-proof documents |
```

20. Viva quick answers

If asked	Say
What is the stack?	Next.js 16 with React 19 and TypeScript for pages and API; a plain-JavaScript rules engine with node:test; Supabase Postgres for data; Vercel for hosting; Gemini for the optional AI checks.
Where is the backend?	In the same project: app/api (routes) → lib/server/service.ts (permissions) → lib/engine (rules). On Vercel each route is a serverless function.
What is the database schema?	Two tables: app_state (id, version, data jsonb, updated_at) holding the whole state as one versioned JSON document, and call_signals for call set-up. Inside the JSON: users, jobs, applications, projects, milestones, SOW documents, messages, task requests, negotiations, work states, check-ins.
How does the frontend reach Supabase?	It does not directly. Browser → lib/api.ts → our API route → persist.ts → Supabase REST with the secret key on the server. Row level security blocks the public key.
How do you avoid lost updates?	Optimistic concurrency: save with "where version = n"; if another request saved first, re-run on fresh data (up to 14 times).
How are the three checks guaranteed?	Section 14: price = hours × rate before tax; only approved/agreed items count; content hashing and same-text detection make repeats harmless. All unit and smoke tested.
Where is the AI and what if it fails?	Gemini only adds a second opinion after the rules; without a key or if it is slow, the rules decide, and AI decisions are labelled.
How is it deployed?	Push to the krishna branch → Vercel builds and serves it; secrets are Vercel environment variables; Supabase holds the data; /api/version shows the live commit.
Is it secure?	Salted scrypt passwords, signed HTTP-only cookie, lockout, server-side validation, role checks on every action, RLS on, secrets only on the server, reset disabled live.